



UNIVERSIDADE VEIGA DE ALMEIDA
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

RELATÓRIO JRC BRASIL

Trabalho A4 de Laboratório de Desenvolvimento de Software

ALAN DIAS DA SILVA - 1240211268
EDUARDO HENRIQUE BRAGA DA SILVA - 1260115910
FELIPE DUMAR BATISTA - 1230103387
GABRIEL MARQUES DOS SANTOS SILVA - 1180104889
IAN ROANITO PEREIRA DA COSTA - 1240210067
JORGE LUIS DE OLIVEIRA FERRARI - 1170103026
KARLA VITORIA FONSECA FERREIRA - 1240206621
LUCAS CALVET DA SILVA DE SOUSA - 1240112772
MATHEUS FARIAS DE OLIVEIRA - 1240112040
MATHEUS OLIVEIRA GOMES - 1240205066
PAULO CARVALHO VIEIRA JUNIOR - 1240211360
VINÍCIUS DE AZEVEDO LIMA E SOUZA - 1240201902

RIO DE JANEIRO

2026

RESUMO

Este trabalho apresenta o desenvolvimento do sistema Relatório JRC Brasil, uma aplicação web criada para automatizar o gerenciamento de ativos de Tecnologia da Informação e a geração de relatórios de compliance exigidos pela matriz da empresa. A solução foi desenvolvida utilizando React no frontend, Django e Django REST Framework no backend, PostgreSQL como banco de dados e Docker para a padronização do ambiente de desenvolvimento. Durante o projeto foram aplicados conceitos de arquitetura cliente-servidor, APIs REST, bancos de dados relacionais, controle de versão com Git e metodologias ágeis. Como resultado, foi obtida uma aplicação funcional capaz de centralizar informações dos ativos de TI, facilitar sua administração e automatizar a geração de relatórios, proporcionando maior organização, confiabilidade e eficiência ao processo.

Palavras-chave: Desenvolvimento Web. Engenharia de Software. API REST. Django. React. PostgreSQL. Docker. Compliance de TI.

LISTA DE ILUSTRAÇÕES

Figura 1	Arquitetura do Sistema	9
Figura 2	Estrutura resumida de arquivos do projeto	10
Figura 3	Fluxo do Sistema	12
Figura 4	DER simplificado das entidades principais do sistema	14
Figura 5	Estrutura base das entidades principais do sistema	15
Figura 6	Interfaces responsivas do sistema	18
Figura 7	Fluxo de telas	19
Figura 8	Tela de Login	20
Figura 9	Tela de Dashboard	20
Figura 10	Tela de Colaboradores	21
Figura 11	Tela de Máquinas	21
Figura 12	Tela de Softwares	22
Figura 13	Tela de Relatórios	22
Figura 14	Épico 0 - Fundação	23
Figura 15	Épico 1 – Frontend	24
Figura 16	Épico 2 – Backend	24
Figura 17	Épico 3 – CRUD Frontend	25
Figura 18	Épico 4 – Infraestrutura	25

LISTA DE TABELAS

Tabela 1	Padrão CRUD	16
Tabela 2	Recursos da API	16
Tabela 3	API de autenticação	17
Tabela 4	API de autenticação	17

SUMÁRIO

1	INTRODUÇÃO	5
2	FUNDAMENTAÇÃO TEÓRICA	6
2.1	DESENVOLVIMENTO DE APLICAÇÕES WEB	6
2.2	ARQUITETURA CLIENTE-SERVIDOR	6
2.3	API REST	7
2.4	BANCO DE DADOS RELACIONAL	7
2.5	FRAMEWORKS PARA DESENVOLVIMENTO WEB	7
2.6	CONTAINERIZAÇÃO	8
2.7	CONTROLE DE VERSÃO	8
2.8	METODOLOGIAS ÁGEIS	8
3	DESENVOLVIMENTO DO SISTEMA	9
3.1	ARQUITETURA DA APLICAÇÃO	9
3.1.1	Arquivo README.md	9
3.1.2	Arquivo CLAUDE.md	10
3.1.3	Arquivo docker-compose.yml	10
3.1.4	Arquivo .gitignore	11
3.1.5	Arquivos .env e .env.*	11
3.2	BACKEND	11
3.2.1	Inicialização e Execução	12
3.3	BANCO DE DADOS	13
3.4	API REST	15
3.5	FRONTEND	17
4	DOCUMENTAÇÃO DAS TELAS	18
4.1	FLUXO DE TELAS	19
4.2	TELA DE LOGIN	20
4.3	TELA DE DASHBOARD	20
4.4	TELA DE COLABORADORES	21
4.5	TELA DE MÁQUINAS	21
4.6	TELA DE SOFTWARES	22
4.7	TELA DE RELATÓRIOS	22
5	ORGANIZAÇÃO ÁGIL E BACKLOG	22

6	VERSIONAMENTO E GITHUB	25
7	CONCLUSÃO	26

1 INTRODUÇÃO

A crescente digitalização de processos e serviços tem impulsionado o desenvolvimento de aplicações web cada vez mais completas, escaláveis e integradas. Nesse contexto, o presente trabalho apresenta o desenvolvimento do sistema "Relatório JRC Brasil", uma aplicação full stack utilizando as tecnologias Django, React, PostgreSQL, Git e Docker, aplicando conceitos estudados na disciplina de Laboratório de Desenvolvimento de Software.

Nossa solução tem como objetivo auxiliar a empresa "JRC Brasil" no gerenciamento e na geração de relatórios de compliance e auditorias de segurança exigidos anualmente pela matriz da empresa localizada no Japão.

Atualmente, o processo de elaboração desses relatórios é realizado de forma manual, envolvendo o levantamento e a organização de informações relacionadas aos ativos de tecnologia da empresa, como computadores, usuários, licenças de software, acessos a servidores e sistemas ERP, antivírus, criptografia e dispositivos corporativos. Esse modelo manual torna o processo suscetível a erros, inconsistências e dificuldades de gerenciamento, além de demandar maior tempo operacional da equipe responsável.

O nosso sistema, com o intuito de automatizar e otimizar esse processo, foi desenvolvido com foco na separação entre frontend e backend, utilizando uma arquitetura baseada em APIs para promover maior organização, modularidade e facilidade de manutenção. O frontend da aplicação foi construído em React, proporcionando uma interface dinâmica e interativa para o usuário, enquanto o backend foi implementado com Django, responsável pelo processamento das regras de negócio, gerenciamento dos dados e comunicação com o banco de dados PostgreSQL.

Além disso, o projeto faz uso da containerização com Docker e versionamento de código com Git, permitindo a padronização do ambiente de desenvolvimento, facilitando a execução da aplicação em diferentes sistemas operacionais e agilizando o processo de desenvolvimento em equipe. Durante o desenvolvimento, foram aplicados conhecimentos relacionados à engenharia de software, modelagem de sistemas, integração entre serviços e desenvolvimento colaborativo.

Dessa forma, este trabalho busca demonstrar não apenas a implementação técnica da aplicação, mas também a aplicação prática de conceitos fundamentais do desenvolvimento moderno de software, evidenciando os desafios encontrados, as soluções adotadas e os resultados alcançados ao longo do projeto.

2 FUNDAMENTAÇÃO TEÓRICA

O desenvolvimento de aplicações corporativas modernas envolve a adoção de arquiteturas e metodologias capazes de atender requisitos como escalabilidade, manutenção, confiabilidade e colaboração entre equipes de desenvolvimento. Nesse contexto, esta seção apresenta os principais conceitos teóricos que fundamentaram o desenvolvimento do sistema Relatório JRC Brasil, abordando aspectos relacionados ao desenvolvimento de aplicações web, arquitetura cliente-servidor, APIs REST, persistência de dados, containerização e práticas modernas de engenharia de software.

2.1 DESENVOLVIMENTO DE APLICAÇÕES WEB

As aplicações web representam uma das principais formas de disponibilização de sistemas de informação nas organizações, permitindo o acesso às funcionalidades do sistema por meio de navegadores, sem a necessidade de instalação de softwares específicos nos dispositivos clientes. Esse modelo reduz custos de manutenção, centraliza o gerenciamento das informações e facilita a atualização contínua da aplicação.

Nos últimos anos, o desenvolvimento web evoluiu para arquiteturas desacopladas, nas quais a interface do usuário e a lógica de negócio são implementadas em aplicações independentes que se comunicam por meio de serviços web. Essa abordagem favorece a reutilização de componentes, simplifica a manutenção do código e permite que diferentes clientes, como aplicações web e dispositivos móveis, consumam os mesmos serviços disponibilizados pelo servidor.

2.2 ARQUITETURA CLIENTE-SERVIDOR

A arquitetura cliente-servidor consiste em um modelo computacional no qual as responsabilidades da aplicação são distribuídas entre clientes, responsáveis pela interação com o usuário, e servidores, responsáveis pelo processamento das requisições, aplicação das regras de negócio e gerenciamento dos dados.

Essa separação promove maior organização estrutural do sistema, reduz o acoplamento entre os componentes da aplicação e facilita sua evolução ao longo do tempo. Além disso, a arquitetura permite que múltiplos clientes utilizem simultaneamente os mesmos serviços disponibilizados pelo servidor, característica essencial para sistemas corporativos.

2.3 API REST

A integração entre os componentes de aplicações modernas é frequentemente realizada por meio de APIs (Application Programming Interfaces). Entre os modelos arquiteturais existentes, destaca-se o REST (Representational State Transfer), proposto por Roy Fielding, que define um conjunto de restrições para comunicação entre sistemas distribuídos.

As APIs REST utilizam o protocolo HTTP para disponibilização de recursos, empregando métodos como GET, POST, PUT, PATCH e DELETE para representar operações de consulta, criação, atualização e remoção de dados. A troca de informações normalmente ocorre utilizando o formato JSON, amplamente adotado devido à sua simplicidade e interoperabilidade entre diferentes linguagens e plataformas.

A utilização de APIs REST favorece o desacoplamento entre frontend e backend, permitindo que cada camada da aplicação evolua de forma independente, desde que sejam preservados os contratos estabelecidos entre os serviços.

2.4 BANCO DE DADOS RELACIONAL

Os Sistemas Gerenciadores de Banco de Dados Relacionais (SGBDR) desempenham papel fundamental em aplicações corporativas ao garantir o armazenamento seguro e consistente das informações. Esses sistemas organizam os dados em tabelas relacionadas entre si por meio de chaves primárias e estrangeiras, promovendo integridade referencial e reduzindo redundâncias.

Além disso, bancos de dados relacionais implementam as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade), essenciais para assegurar a confiabilidade das transações e preservar a consistência dos dados mesmo diante de falhas durante sua execução.

2.5 FRAMEWORKS PARA DESENVOLVIMENTO WEB

Frameworks são conjuntos de bibliotecas e ferramentas que fornecem uma estrutura padronizada para o desenvolvimento de aplicações, permitindo maior produtividade, reutilização de código e padronização das soluções implementadas.

No desenvolvimento backend, frameworks como Django oferecem recursos integrados para autenticação, gerenciamento de banco de dados, controle de permissões e implementação de APIs, reduzindo a necessidade de desenvolvimento de funcionalidades comuns. No frontend, bibliotecas baseadas em componentes, como React, favorecem a construção de interfaces reuti-

lizáveis, facilitando a manutenção e evolução da aplicação.

2.6 CONTAINERIZAÇÃO

A containerização consiste em uma técnica de virtualização em nível de sistema operacional que permite empacotar uma aplicação juntamente com todas as suas dependências, garantindo que ela seja executada da mesma forma em diferentes ambientes computacionais.

Essa abordagem reduz problemas relacionados à configuração do ambiente de desenvolvimento, facilita a implantação da aplicação e aumenta a portabilidade entre diferentes plataformas. Além disso, ferramentas de orquestração permitem executar múltiplos serviços de forma integrada, como servidores de aplicação e bancos de dados.

2.7 CONTROLE DE VERSÃO

O controle de versão é uma prática essencial no desenvolvimento de software, permitindo registrar e acompanhar todas as alterações realizadas no código-fonte ao longo do ciclo de desenvolvimento. Sistemas distribuídos de controle de versão possibilitam que múltiplos desenvolvedores trabalhem simultaneamente no mesmo projeto, mantendo o histórico das modificações e facilitando a integração das contribuições realizadas por cada integrante da equipe.

Além de favorecer o desenvolvimento colaborativo, o controle de versão possibilita a recuperação de versões anteriores do sistema, a criação de ramificações para implementação de novas funcionalidades e o gerenciamento das diferentes etapas de evolução do software.

2.8 METODOLOGIAS ÁGEIS

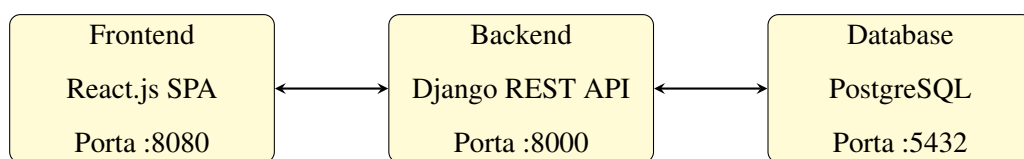
As metodologias ágeis constituem um conjunto de práticas voltadas ao desenvolvimento incremental e colaborativo de software, priorizando entregas frequentes, comunicação entre os membros da equipe e adaptação contínua às mudanças de requisitos.

Entre os principais mecanismos utilizados destaca-se o backlog do produto, responsável por organizar as funcionalidades e atividades previstas para o projeto de acordo com sua prioridade. Essa abordagem permite maior controle sobre a evolução do desenvolvimento, facilita a distribuição das tarefas entre os integrantes da equipe e possibilita acompanhar o progresso do projeto por meio de entregas sucessivas de software funcional.

3 DESENVOLVIMENTO DO SISTEMA

O sistema “Relatório JRC Brasil” foi projetada com o objetivo de automatizar o processo de geração e gerenciamento de relatórios de compliance de TI, permitindo o cadastro centralizado e o tratamento das informações relacionadas aos ativos tecnológicos da empresa. Nesse sistema, foram empregadas tecnologias e práticas modernas amplamente utilizadas no mercado de software, abordadas ao longo da disciplina de Laboratório de Desenvolvimento de Software.

Figura 1: Arquitetura do Sistema



Fonte: Elaborado pelos autores (2026).

3.1 ARQUITETURA DA APLICAÇÃO

A aplicação foi implementada utilizando Django no backend, React no frontend, PostgreSQL como sistema gerenciador de banco de dados e Docker para a padronização do ambiente de desenvolvimento e execução da aplicação. Além disso, foram utilizadas práticas de desenvolvimento colaborativo por meio do versionamento de código com Git e da utilização de metodologias ágeis para organização das atividades da equipe e acompanhamento do progresso do projeto. A comunicação entre os módulos do sistema foi realizada por meio de uma API REST, responsável pela troca de informações entre a interface do usuário e o servidor.

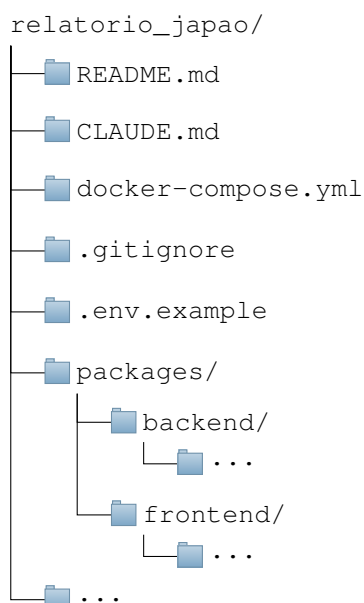
Como é possível observar pela figura 2, a estrutura do projeto é organizada em uma hierarquia de arquivos e pastas que reflete a separação entre as camadas de frontend e backend, bem como os arquivos de configuração e documentação do projeto. Essa organização facilita a navegação e o entendimento do código, além de promover uma melhor manutenção e escalabilidade da aplicação ao longo do tempo.

Ainda seguindo a figura 2, é necessário destacar alguns dos arquivos e pastas mais relevantes para o projeto, pois o entendimento deles destaca boas práticas de desenvolvimento de software.

3.1.1 Arquivo README.md

O arquivo README.md é um documento essencial para qualquer projeto de software, pois serve como a porta de entrada para os desenvolvedores e usuários que desejam entender o

Figura 2: Estrutura resumida de arquivos do projeto



Fonte: Elaborado pelos autores (2026).

propósito, a estrutura e as instruções de uso do sistema. Ele deve conter informações claras e concisas sobre o projeto, incluindo uma descrição geral, requisitos de instalação, instruções de configuração, exemplos de uso e detalhes sobre a arquitetura do sistema.

3.1.2 Arquivo CLAUDE.md

As LLMs, comunmente conhecidas como "IA", estão cada vez mais presentes no desenvolvimento de software, oferecendo suporte em diversas etapas do processo, almentando a produtividade e a qualidade do código.

Nesse projeto utilizamos do Claude, um modelo de linguagem avançado desenvolvido pela Anthropic, para auxiliar na geração de código, documentação e outras tarefas relacionadas ao desenvolvimento de software. O arquivo CLAUDE.md contém informações sobre como o modelo foi utilizado no projeto, incluindo exemplos de prompts e respostas geradas pelo modelo, bem como detalhes sobre a integração do Claude com as ferramentas de desenvolvimento utilizadas pela equipe.

3.1.3 Arquivo docker-compose.yml

O arquivo `docker-compose.yml` é responsável por definir os serviços necessários para a execução da aplicação em um ambiente Docker. Ele especifica as imagens, volumes, redes e outras configurações necessárias para criar e gerenciar os contêineres que compõem a aplicação. Esse

arquivo é fundamental para garantir a portabilidade e a consistência do ambiente de desenvolvimento e produção, permitindo que a aplicação seja inicializada de forma rápida e consistente, independente da máquina host onde o ambiente esteja rodando.

3.1.4 Arquivo .gitignore

O Git é uma ferramenta de controle de versão amplamente utilizada no desenvolvimento de software, permitindo que os desenvolvedores acompanhem as mudanças no código-fonte ao longo do tempo. O arquivo `.gitignore` é utilizado para especificar quais arquivos ou pastas devem ser ignorados pelo Git, ou seja, não devem ser incluídos no repositório de código. Isso é especialmente útil para evitar o versionamento de arquivos temporários, arquivos de configuração local, dependências e outros arquivos que não são relevantes para o desenvolvimento colaborativo do projeto.

3.1.5 Arquivos .env e .env.*

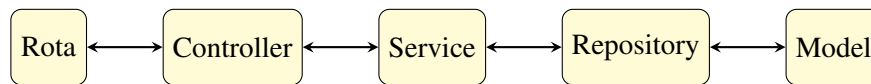
Qualquer projeto de software moderno vai precisar configurar aspectos como conexões de banco de dados, chaves de API, configurações de ambiente (desenvolvimento, produção, etc.) e outras informações sensíveis. Escrever essas informações diretamente no código seria pouco prático e inseguro. O arquivo `.env` é utilizado para armazenar essas variáveis de ambiente de forma segura e organizada, permitindo que a aplicação acesse essas informações durante a execução. O arquivo `.env.example` serve como um modelo para a configuração das variáveis de ambiente, fornecendo um guia para os desenvolvedores sobre quais variáveis são necessárias e como devem ser configuradas.

3.2 BACKEND

O backend da aplicação foi construído sobre Django 4.2 com Django REST Framework (DRF), expondo uma API REST que serve como única fonte de dados para o frontend React. A escolha pelo Django se justifica pelo domínio do problema: o sistema precisa gerenciar múltiplos relatórios de auditoria de compliance de TI, com diversas entidades relacionadas. O ORM do Django, somado ao Django Admin e ao ecossistema do DRF, acelera o desenvolvimento de operações CRUD robustas sem abrir mão de validação e segurança.

A organização do código segue uma arquitetura em camadas com responsabilidade única bem definida, de acordo com o fluxo na figura 3.

Figura 3: Fluxo do Sistema



Fonte: Elaborado pelos autores (2026).

O Controller (`controllers.py`) lida apenas com o ciclo HTTP. Ele recebe a requisição, valida permissões e devolve a resposta, delegando toda a lógica de negócio para o Service (`services.py`). O Service orquestra as operações e usa transações atômicas quando há passos dependentes (por exemplo, criar um colaborador junto de seus e-mails). O acesso ao banco é isolado no Repository (`repositories.py`), que concentra as queries do ORM, enquanto o Serializer cuida da validação de entrada e da formatação de saída. Esse desenho aplica princípios como SOLID, a Lei de Demeter e o "Tell, Don't Ask", evitando que regras de negócio vazem para os controllers ou que os controllers conversem diretamente com o banco.

Entre as principais features do backend estão:

- Autenticação JWT, com tokens de acesso e refresh, rotação e blacklist no logout;
- Soft delete centralizado, de modo que nenhum registro é fisicamente apagado, apenas marcado com "deleted_at", preservando o histórico exigido por uma aplicação de auditoria;
- Paginação, filtros, busca e ordenação padronizados nos endpoints;
- CORS habilitado para a comunicação com o SPA;
- Exportação dos relatórios em PDF e Excel, usando as bibliotecas `reportlab` e `openpyxl`.

3.2.1 Inicialização e Execução

A aplicação é orquestrada pelo Docker Compose, que sobe três contêineres:

- O banco PostgreSQL
- O backend Django
- O frontend React

O `docker-compose.yml` declara as dependências entre eles, em especial, o backend só inicia depois que o PostgreSQL passa por um `healthcheck`, eliminando o acoplamento temporal clássico de "subir o banco e a aplicação ao mesmo tempo", em que a API tentaria conectar antes do banco estar pronto.

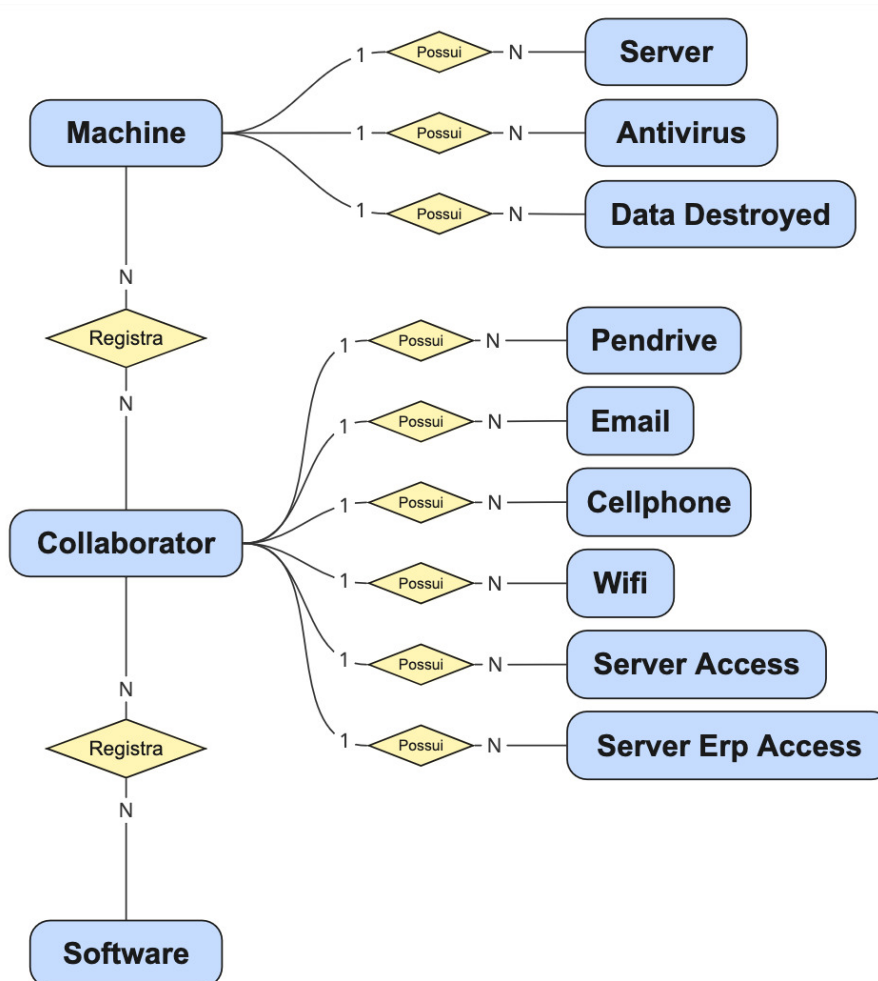
O fluxo de construção do backend começa pelo arquivo "`Dockerfile`", que inicia a partir de uma imagem que instala o python 3 em uma máquina linux, instala o cliente do PostgreSQL e as dependências Python listadas no `requirements.txt`, e então executa o script `entrypoint.sh` que, em ordem explícita: aguarda em loop até o PostgreSQL responder; aplica as migrations, cria o esquema do banco a partir dos modelos Django; carrega dados iniciais de exemplo caso ainda não existam; cria um superusuário de desenvolvimento se não houver; e sobe o servidor com `runserver` na porta 8000.

Em resumo, basta um "`docker-compose up build`" para ter o ambiente completo, banco migrado, dados de exemplo e API no ar, sem nenhum passo manual.

3.3 BANCO DE DADOS

O banco de dados da aplicação foi desenvolvido utilizando PostgreSQL, sendo responsável pelo armazenamento, organização e gerenciamento das informações utilizadas pelo sistema. A modelagem foi estruturada de forma a garantir integridade, consistência e relacionamento adequado entre os dados.

Figura 4: DER simplificado das entidades principais do sistema



Fonte: elaborado pelos autores (2026)

Como exibido na figura 4, a estrutura do banco foi projetada com base nas entidades necessárias para o gerenciamento dos ativos de tecnologia da informação da empresa, incluindo informações relacionadas a usuários, equipamentos, softwares, acessos, licenças e demais recursos utilizados na geração dos relatórios de compliance.

Em sua implementação, as entidades principais receberam o prefixo "CORE_" para indicar que são as entidades centrais do sistema. Além disso, vale notar que todas as entidades principais compartilham da mesma estrutura base, assim como descrito na figura 5, composta pelos atributos de identificação, data de criação, data de atualização e data de exclusão, respeitando boas práticas de modelagem e garantindo a integridade dos dados no caso de alterações futuras.

Figura 5: Estrutura base das entidades principais do sistema

ENTITY_NAME			
id	bigint	Identity	PK
created_at	timestampz	Not Null	
updated_at	timestampz	Not Null	
deleted_at	timestampz		
...			

Fonte: elaborado pelos autores (2026)

Durante o desenvolvimento, a utilização das migrations do Django em conjunto com a containerização através do Docker permitiu a padronização e inicialização consistente do banco de dados entre todos os integrantes da equipe. Essa abordagem possibilitou que a estrutura das tabelas e relacionamentos fosse criada e atualizada automaticamente em diferentes ambientes de desenvolvimento, reduzindo inconsistências e facilitando o processo colaborativo.

3.4 API REST

A comunicação entre o frontend e o backend acontece exclusivamente por meio de uma API REST servida pelo Django REST Framework. Todos os recursos ficam sob o prefixo */api/*, retornam e recebem um pacote JSON. Com as exceções de autenticação e o healthcheck, todas as requisições exigem um token JWT válido no cabeçalho. As rotas são distribuídas em três grupos funcionais:

- Autenticação */api/auth/*
- Entidades de negócio */api/, app core*
- Relatórios de compliance */api/reports/, app reports*

Padrão CRUD dos recursos de negócio

Tabela 1: Padrão CRUD

Método	Caminho	Função
GET	/api/<recurso>/	Lista registros (paginado, com filtros, busca e ordenação)
POST	/api/<recurso>/	Cria um novo registro
GET	/api/<recurso>/{id}/	Recupera um registro específico pelo ID
PUT	/api/<recurso>/{id}/	Atualiza integralmente um registro
PATCH	/api/<recurso>/{id}/	Atualiza parcialmente um registro
DELETE	/api/<recurso>/{id}/	Remove (soft delete) um registro

Fonte: elaborado pelos autores (2026)

Os 12 recursos que seguem esse padrão são:

Tabela 2: Recursos da API

Recurso	Caminho base	Entidade
Colaboradores	/api/collaborators/	Collaborator
Máquinas	/api/machines/	Machine
Softwares	/api/software/	Software
E-mails	/api/emails/	Email
Celulares	/api/cellphones/	Cellphone
WiFi	/api/wifi/	Wifi
Antivírus	/api/antivirus/	AntiVirus
Servidores	/api/servers/	Server
Acesso a servidor	/api/server-access/	ServerAccess
Acesso ao ERP	/api/erp-access/	ServerErpAccess
Destruição de dados	/api/data-destroyed/	DataDestroyed
Pendrives	/api/pen-drives/	PenDrive

Fonte: elaborado pelos autores (2026)

O login e o refresh são as únicas rotas verdadeiramente públicas da API e o register é deliberadamente restrito a administradores, impedindo auto-cadastro.

Tabela 3: API de autenticação

Método	Caminho	Função	Permissão
POST	/api/auth/login/	Autentica e devolve par de tokens	Pública
POST	/api/auth/refresh/	Renova o token de acesso	Pública
POST	/api/auth/logout/	Invalida o refresh token via blacklist	Autenticado
GET	/api/auth/me/	Retorna os dados do usuário logado	Autenticado
POST	/api/auth/register/	Cria um novo usuário	Apenas admin

Fonte: elaborado pelos autores (2026)

Existeme três rotas voltadas aos 19 relatórios de auditoria e compliance, definidas como:

Tabela 4: API de autenticação

Método	Caminho	Função
GET	/api/reports/	Lista os 19 relatórios com seus metadados
POST	/api/reports/{number}/generate/	Marca um relatório como gerado
GET	/api/reports/{number}/?format=pdf lsx	Exportação do relatório em PDF ou Excel

Fonte: elaborado pelos autores (2026)

3.5 FRONTEND

O frontend da aplicação foi desenvolvido utilizando a framework React.js, sendo responsável pela construção da interface gráfica e pela interação entre o usuário e o sistema. A utilização de uma arquitetura baseada em componentes permitiu maior reutilização de código, organização da interface e facilidade na manutenção das funcionalidades implementadas.

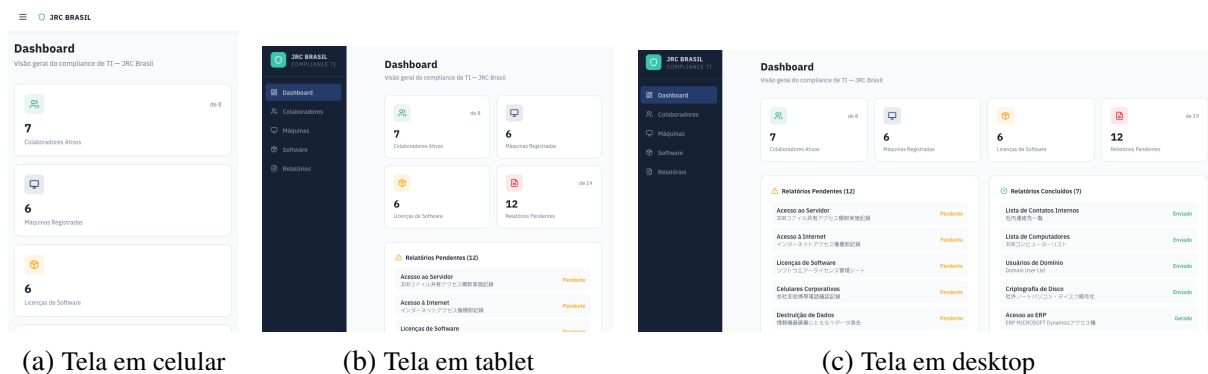
A estrutura do frontend foi organizada de forma modular, separando páginas, componentes, serviços e rotas da aplicação. Essa abordagem contribuiu para uma melhor divisão de responsabilidades dentro do projeto, além de facilitar a integração com o backend através das requisições realizadas à API REST da aplicação.

Durante o desenvolvimento da interface foram implementadas funcionalidades relacionadas ao gerenciamento dos ativos de tecnologia da informação, incluindo telas de cadastro, listagem, autenticação e visualização de informações utilizadas na geração dos relatórios de compliance. O frontend também foi projetado buscando oferecer uma navegação intuitiva e uma experiência de uso consistente para os usuários do sistema.

4 DOCUMENTAÇÃO DAS TELAS

Cada tela foi projetada buscando facilitar a interação do usuário com a aplicação, permitindo o gerenciamento das informações de forma organizada e intuitiva. Nas subseções seguintes são apresentadas capturas de tela acompanhadas de descrições resumidas sobre a finalidade e o funcionamento de cada interface implementada no sistema.

Figura 6: Interfaces responsivas do sistema



(a) Tela em celular

(b) Tela em tablet

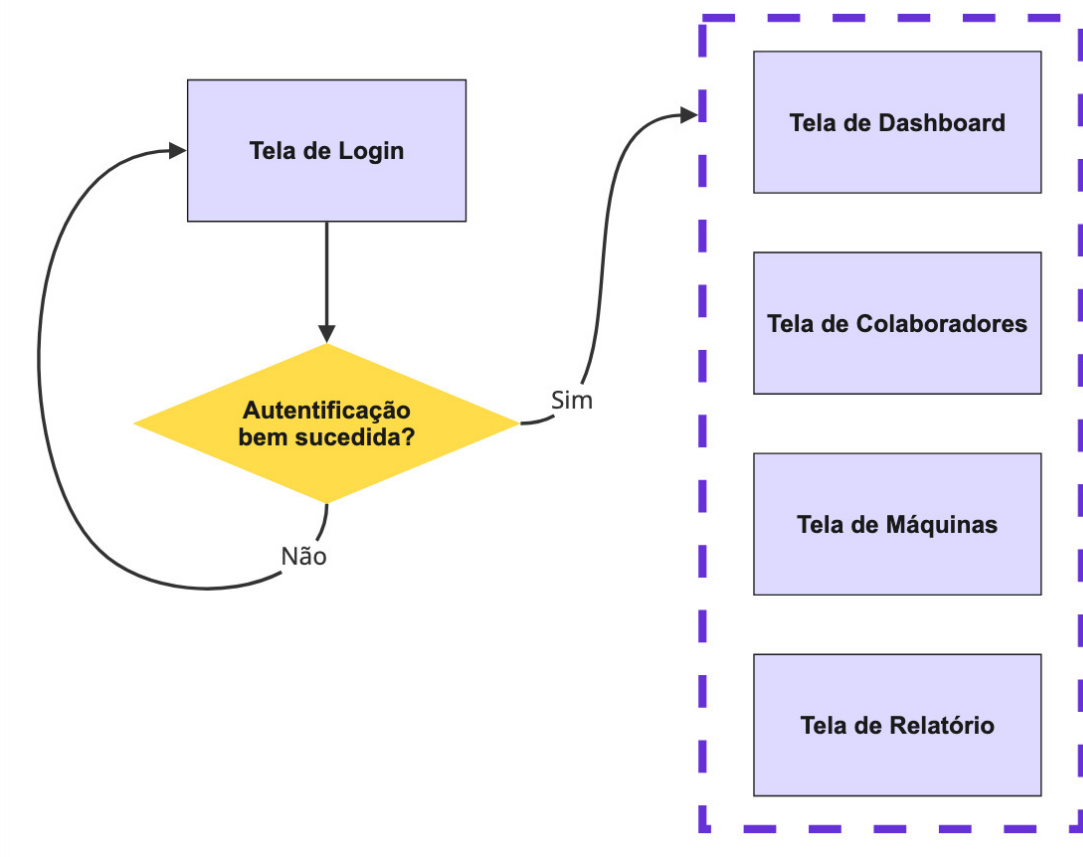
(c) Tela em desktop

Fonte: elaborado pelos autores (2026)

Como demonstrado pela figura 6, a interface do sistema foi projetada para ser responsiva, adaptando-se de forma eficiente a diferentes tamanhos de tela, desde dispositivos móveis até desktops. Essa abordagem visa garantir uma experiência de uso consistente e acessível para os usuários, independentemente do dispositivo utilizado para acessar a aplicação.

4.1 FLUXO DE TELAS

Figura 7: Fluxo de telas

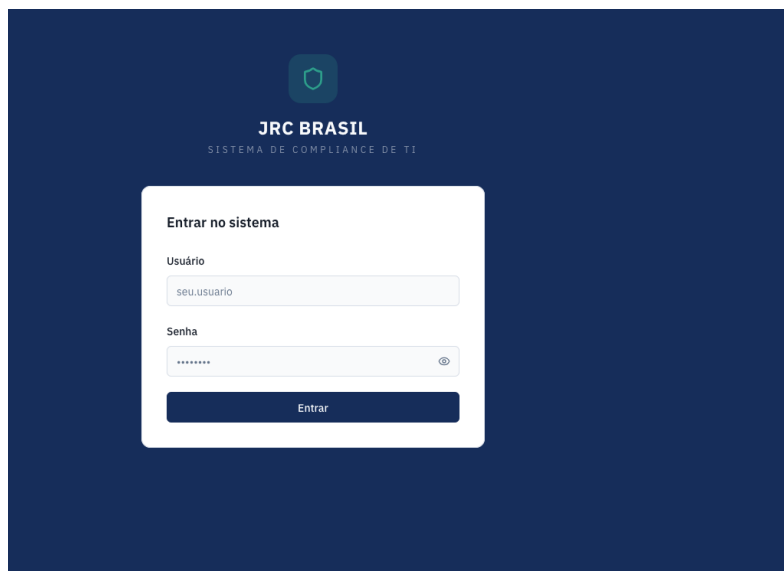


Fonte: elaborado pelos autores (2026)

A figura 7 apresenta o fluxo de telas da aplicação, ilustrando a navegação entre as diferentes interfaces implementadas no sistema. Como é possível observar, todo usuário deve passar pela tela de login para acessar as funcionalidades do sistema. A partir da tela de login, os usuários autenticados podem navegar para as telas de dashboard, de colaboradores, de máquinas e de relatórios.

4.2 TELA DE LOGIN

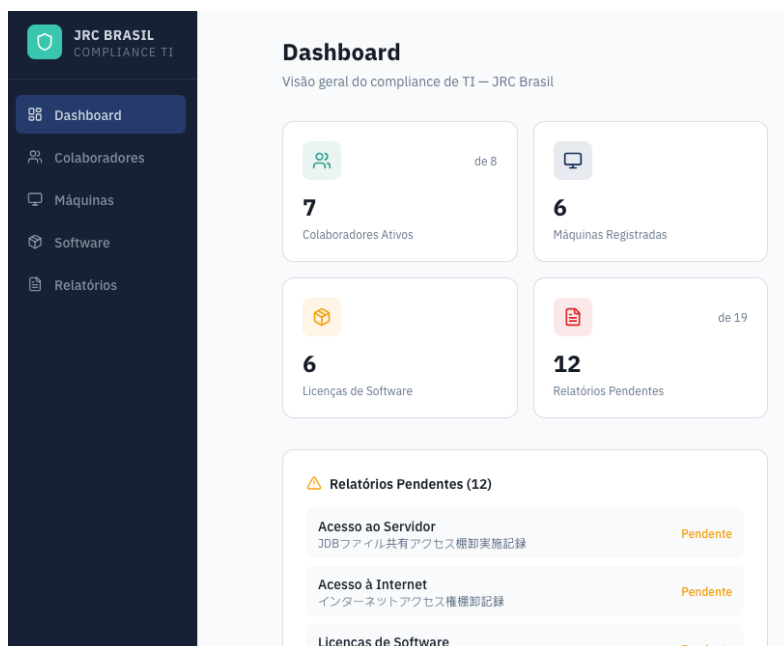
Figura 8: Tela de Login



Fonte: elaborado pelos autores (2026)

4.3 TELA DE DASHBOARD

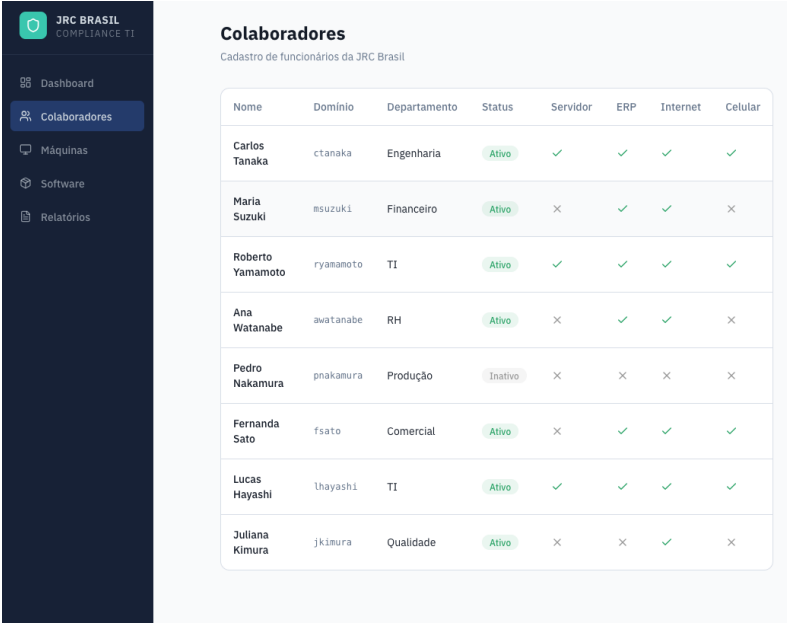
Figura 9: Tela de Dashboard



Fonte: elaborado pelos autores (2026)

4.4 TELA DE COLABORADORES

Figura 10: Tela de Colaboradores

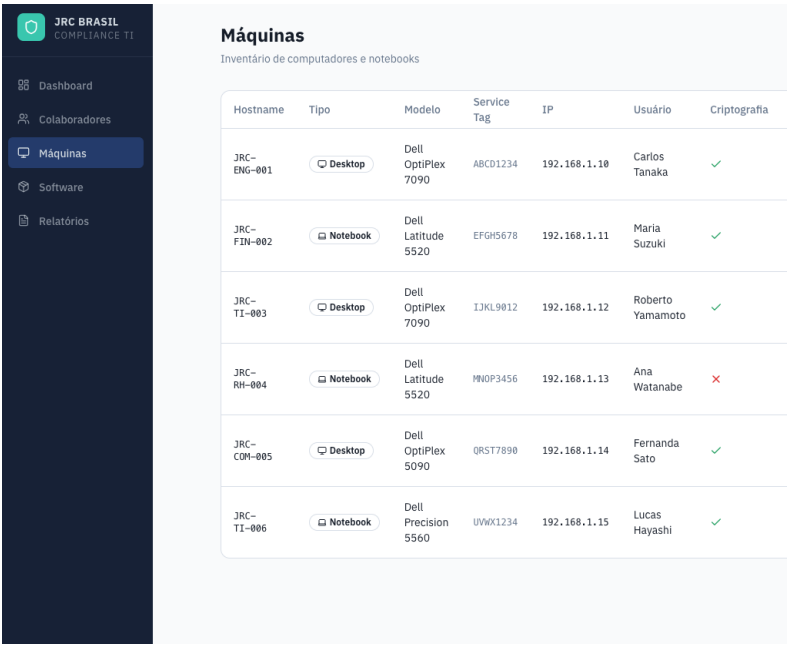


Nome	Domínio	Departamento	Status	Servidor	ERP	Internet	Celular
Carlos Tanaka	ctanaka	Engenharia	Ativo	✓	✓	✓	✓
Maria Suzuki	msuzuki	Financeiro	Ativo	X	✓	✓	X
Roberto Yamamoto	ryamamoto	TI	Ativo	✓	✓	✓	✓
Ana Watanabe	awatanabe	RH	Ativo	X	✓	✓	X
Pedro Nakamura	pnakamura	Produção	Inativo	X	X	X	X
Fernanda Sato	fsato	Comercial	Ativo	X	✓	✓	✓
Lucas Hayashi	lhayashi	TI	Ativo	✓	✓	✓	✓
Juliana Kimura	jkimura	Qualidade	Ativo	X	X	✓	X

Fonte: elaborado pelos autores (2026)

4.5 TELA DE MÁQUINAS

Figura 11: Tela de Máquinas

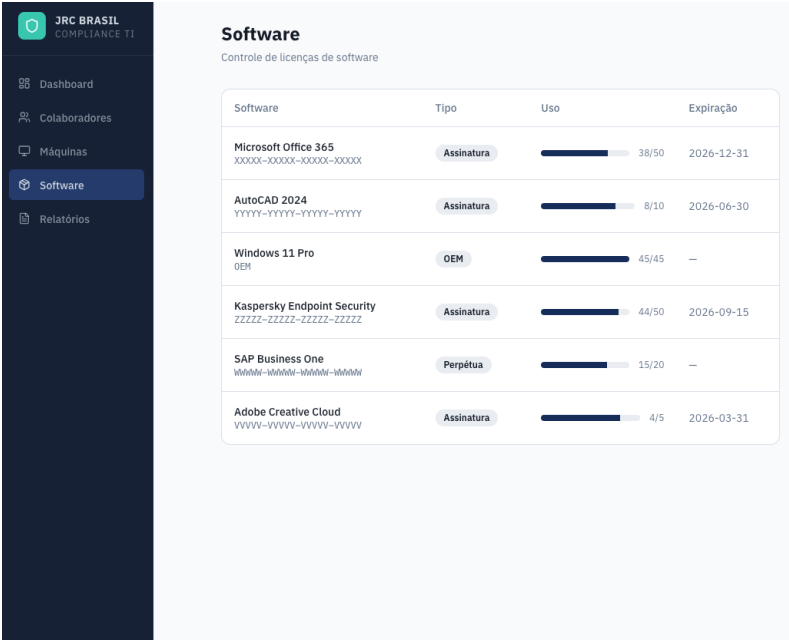


Hostname	Tipo	Modelo	Service Tag	IP	Usuário	Criptografia
JRC-ENG-001	Desktop	Dell OptiPlex 7090	ABCD1234	192.168.1.10	Carlos Tanaka	✓
JRC-FIN-002	Notebook	Dell Latitude 5520	EFGH5678	192.168.1.11	Maria Suzuki	✓
JRC-TI-003	Desktop	Dell OptiPlex 7090	IJKL9012	192.168.1.12	Roberto Yamamoto	✓
JRC-RH-004	Notebook	Dell Latitude 5520	MNOP3456	192.168.1.13	Ana Watanabe	X
JRC-COM-005	Desktop	Dell OptiPlex 5090	QRST7890	192.168.1.14	Fernanda Sato	✓
JRC-TI-006	Notebook	Dell Precision 5560	UVWX1234	192.168.1.15	Lucas Hayashi	✓

Fonte: elaborado pelos autores (2026)

4.6 TELA DE SOFTWARES

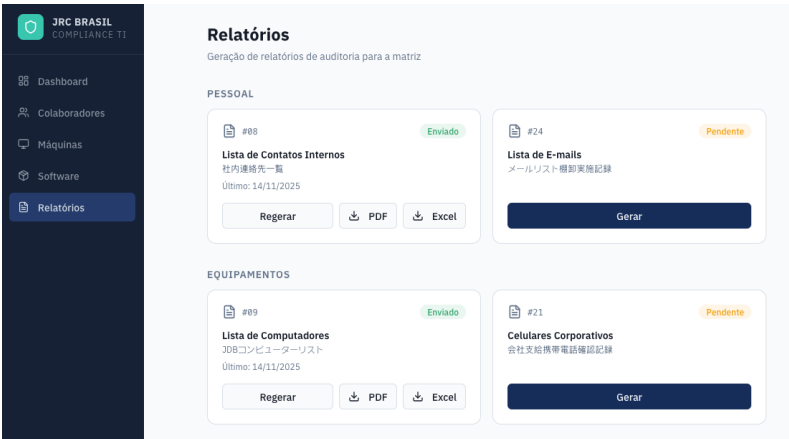
Figura 12: Tela de Softwares



Fonte: elaborado pelos autores (2026)

4.7 TELA DE RELATÓRIOS

Figura 13: Tela de Relatórios



Fonte: elaborado pelos autores (2026)

5 ORGANIZAÇÃO ÁGIL E BACKLOG

Durante o desenvolvimento do sistema foi adotada uma metodologia ágil para organizar as atividades da equipe e acompanhar a evolução do projeto. A utilização dessa abordagem

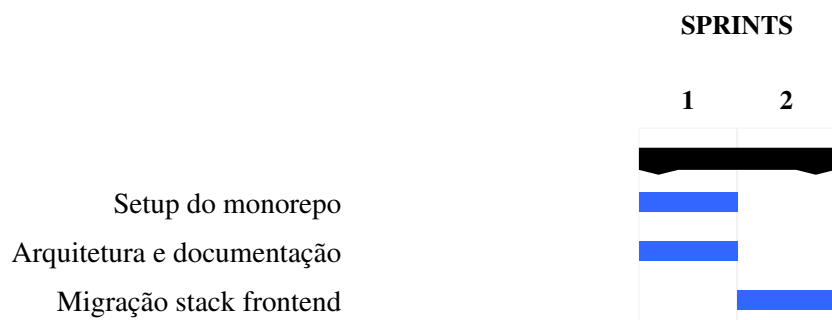
permitiu dividir o desenvolvimento em pequenas entregas, facilitando o planejamento das funcionalidades, o acompanhamento do progresso e a adaptação às necessidades identificadas ao longo da implementação.

Como ferramenta de gerenciamento das atividades foi utilizado um backlog do projeto, no qual foram registradas as funcionalidades, correções e melhorias previstas para o sistema. Cada item do backlog representava uma tarefa a ser desenvolvida e era organizado de acordo com sua prioridade e estado de execução, proporcionando uma visão clara do andamento do projeto.

A organização das tarefas também favoreceu a divisão das responsabilidades entre os integrantes da equipe, permitindo que o desenvolvimento ocorresse de forma paralela e colaborativa. À medida que novas funcionalidades eram concluídas ou novas demandas surgiam, o backlog era atualizado para refletir o estado atual do projeto, mantendo o planejamento alinhado com o desenvolvimento da aplicação.

As figuras 14 à 18 apresentam o backlog utilizado durante o desenvolvimento do sistema, ilustrando a organização das atividades e o fluxo de trabalho realizado pela equipe.

Figura 14: Épico 0 - Fundação



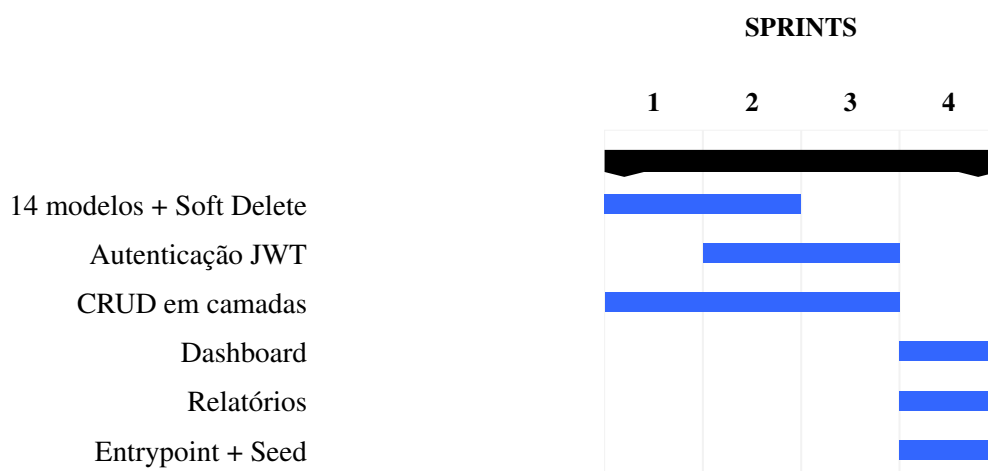
Fonte: elaborado pelos autores (2026)

Figura 15: Épico 1 – Frontend



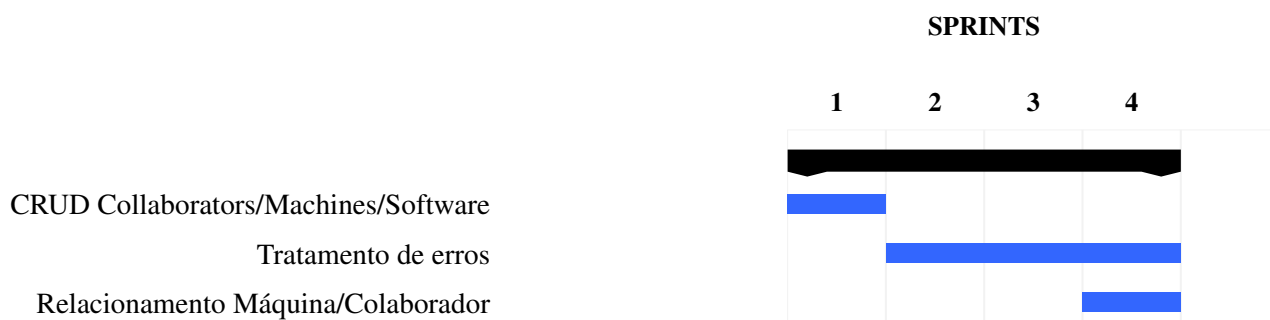
Fonte: elaborado pelos autores (2026)

Figura 16: Épico 2 – Backend



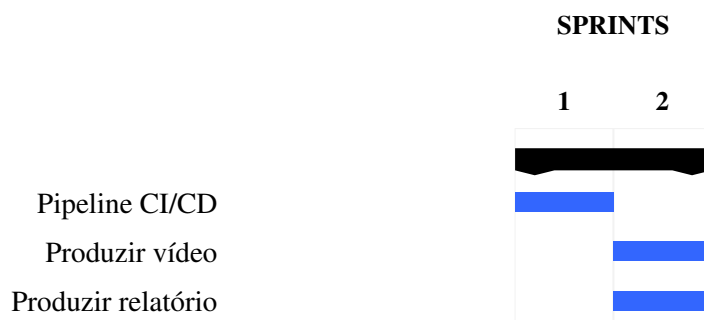
Fonte: elaborado pelos autores (2026)

Figura 17: Épico 3 – CRUD Frontend



Fonte: elaborado pelos autores (2026)

Figura 18: Épico 4 – Infraestrutura



Fonte: elaborado pelos autores (2026)

6 VERSIONAMENTO E GITHUB

O controle de versão do projeto foi realizado por meio do software Git, permitindo o gerenciamento das alterações realizadas ao longo do desenvolvimento da aplicação. A utilização dessa ferramenta possibilitou o trabalho colaborativo entre os integrantes da equipe, garantindo o registro do histórico de modificações, o acompanhamento da evolução do código-fonte e a integração das contribuições de cada desenvolvedor de forma organizada.

O repositório do projeto foi hospedado na plataforma GitHub, que serviu como ambiente centralizado para armazenamento do código, compartilhamento das alterações e gerenciamento das versões da aplicação. Além de facilitar o desenvolvimento colaborativo, a plataforma permitiu a rastreabilidade das modificações por meio de commits e branches, contribuindo para a manutenção da qualidade e da organização do projeto.

O código-fonte completo do sistema encontra-se disponível no repositório oficial do projeto, acessível pelo endereço: https://github.com/ChewieSoft/relatorio_japao

7 CONCLUSÃO

O desenvolvimento do sistema Relatório JRC Brasil possibilitou a aplicação prática dos conceitos abordados na disciplina de Laboratório de Desenvolvimento de Software, integrando conhecimentos de desenvolvimento web, banco de dados, arquitetura de software, APIs REST, controle de versão e metodologias ágeis em um único projeto. A utilização das tecnologias Django, React, PostgreSQL, Docker e Git permitiu a construção de uma aplicação full stack moderna, organizada e adequada às necessidades propostas pelo estudo de caso.

Como resultado, foi desenvolvida uma aplicação capaz de centralizar o cadastro de informações relacionadas aos ativos de tecnologia da informação da empresa e automatizar o gerenciamento dos relatórios de compliance exigidos pela matriz da JRC Brasil. A adoção de uma arquitetura desacoplada entre frontend e backend, juntamente com a utilização de uma API REST, proporcionou maior modularidade e facilitou o desenvolvimento e a manutenção da aplicação.

Além dos resultados técnicos, o projeto contribuiu significativamente para a formação acadêmica dos integrantes da equipe, permitindo a vivência de práticas amplamente empregadas no desenvolvimento profissional de software, como o trabalho colaborativo utilizando Git, a organização das atividades por meio de metodologias ágeis e o desenvolvimento de aplicações containerizadas com Docker.

Conclui-se que os objetivos propostos para o projeto foram alcançados, resultando em uma solução funcional e alinhada às necessidades identificadas. Como trabalhos futuros, podem ser consideradas melhorias relacionadas à ampliação das funcionalidades do sistema, implementação de mecanismos avançados de segurança, geração de novos modelos de relatórios, integração com sistemas corporativos e disponibilização da aplicação em ambiente de produção, ampliando sua utilização em cenários reais.